

Multi-Asset Financial Directional Forecasting using Deep Learning: An End-to-End Framework Review



BCSE306L

Artificial Intelligence

Fall Semester 2025-26

Multi-Asset Financial Directional Forecasting using Deep Learning: An End-to-End Framework Review

Project Team:

23BCE2120 Shree Lakshmi
23BCE2248 Madhav Nanda
23BCE2249 Keshav Nanda
23BCE2275 Dhanalakshmi S

Abstract

This project establishes a comprehensive, end-to-end framework for predicting the directional movement of multi-asset financial time series using deep learning techniques. The objective was to design, implement, and deploy a robust neural network model capable of classifying the next-day price movement (up or down) for a selected portfolio of four large-cap technology stocks: Apple (AAPL), Microsoft (MSFT), Alphabet (GOOGL), and Amazon (AMZN). The core innovation lies in utilizing an **Attention-Mechanism-enhanced Bidirectional Long Short-Term Memory (Attention BiLSTM)** network, which processes concatenated features across all four assets simultaneously to exploit cross-correlation effects. To counter the inherent class imbalance of financial data, the model utilizes **Focal Loss** during training. Performance evaluation focuses on classification metrics critical for imbalanced financial data, including the F1-Score, and Directional Accuracy (DA). The final deliverable includes a complete MLOps pipeline, demonstrating data ingestion, model training, and low-latency inference deployment via a **Gradio** web interface, laying the groundwork for integration into a live algorithmic trading system. The model achieves a mean out-of-sample Directional Accuracy of **66.67%**, significantly outperforming the 50% random baseline across the volatile test period.

1. Introduction

1.1 Background and Motivation

Financial markets are recognized as **complex, adaptive, and non-stationary dynamical systems**. Their behavior is characterized by high **volatility**, inherent **noise**, and emergent properties driven by human psychology and global events. Accurately forecasting stock price movements—specifically the directional change—remains the foundational and most challenging problem in quantitative finance.

Traditional forecasting attempts have historically relied on **linear econometric models** such as Autoregressive Integrated Moving Average (**ARIMA**) and Generalized Autoregressive Conditional Heteroskedasticity (**GARCH**). While these models are effective for capturing simple mean-reversion or variance clustering, they fundamentally fail to account for the crucial **non-linear dependencies** and long-term memory effects embedded within modern price data. These effects are often driven by high-frequency trading algorithms, complex derivative instruments, **macroeconomic factors**, and the intricate web of **inter-stock relationships** that bind the market. This limitation renders traditional, single-factor models sub-optimal for achieving a stable predictive edge.

The current paradigm shift is powered by **Deep Learning (DL)**, which provides the architectural complexity necessary to model these intricate temporal dependencies. By employing **Recurrent Neural Networks (RNNs)** and their variants, like the Long Short-Term Memory (LSTM), we can effectively capture long-range patterns and context that influence daily movements. Furthermore, this project advances beyond a single-asset view to a **multi-asset approach**. We recognize that major technology stocks (AAPL, MSFT, GOOGL, AMZN) are highly **correlated and mutually informative**, operating as intertwined components of the broader technology sector. Modeling them concurrently allows the network to exploit these cross-correlation effects, leading to a more robust and informed prediction. Finally, this project emphasizes the critical need for a **full MLOps framework** to bridge the gap between theoretical model development and reliable, real-world operational deployment.

1.2 Project Objective and Scope

The principal aim of this project is to establish a comprehensive, end-to-end operational framework for multi-asset financial time series prediction using an advanced deep learning methodology. Our focus is on achieving a statistically significant directional predictive edge that is robust against market noise and volatility.

Specific Objectives:

1. **Data Curation and Feature Engineering:**
 - Collect and process daily historical OHLCV data for the four target equities (AAPL, MSFT, GOOGL, ANZM) from 2018 onwards.
 - Compute and integrate a comprehensive set of **15 technical indicators (TIs)**, including various Moving Averages (MA/EMA), Momentum, Volatility, and the Relative Strength Index (RSI), ensuring the feature space is sufficiently rich.
 - Implement **per-ticker Standard Scaling** to normalize features while preserving the unique statistical characteristics of each asset, a crucial step for multi-asset modeling.
2. **Advanced Modeling Architecture:**
 - Design and implement an **Attention-Mechanism-enhanced Bidirectional LSTM (Attention BiLSTM)** network using PyTorch. This architecture is specifically chosen for its capability to capture long-term sequence dependencies (**BiLSTM**) and dynamically assign importance to the most relevant historical time steps (**Attention**), providing both performance and a degree of interpretability.
3. **Robust Classification Formulation:**
 - Define the objective as a binary classification task: predicting $y_{t+1} \in \{0, 1\}$, where 1 denotes an "Up" movement (Close price increases) and 0 denotes an "Down" movement (Close price stays the same or decreases).
 - Address the inherent class imbalance of financial data by utilizing **Focal Loss** during training, which dynamically adjusts weights to prioritize learning from difficult-to-classify examples, thereby enhancing robustness.
4. **Rigorous Evaluation and Backtesting:**
 - Rigorously evaluate model performance using financial-specific metrics, primarily **Directional Accuracy (DA)**, alongside standard classification measures (Precision, F1-Score), and perform a simulated cumulative returns backtest to assess real-world viability.
5. **Operational Deployment (MLOps):**
 - Implement a clean, containerized, and low-latency API endpoint using the **Gradio** framework for instant, real-time inference. This step demonstrates a fully realized, production-ready solution suitable for integration into a larger algorithmic trading platform.

2. Problem Statement

The core challenge of this project is to build a model $\mathcal{F}$ such that, given a historical sequence of multi-asset features up to time t , it can accurately forecast the directional movement y_{t+1} for the next trading day.

2.1 Formalization of the Prediction Task

The task can be formalized as finding a mapping F from an input sequence of length $\mathcal{L}$ to a binary output:

Where:

- $L=21$ is the lookback window (the past 21 trading days).
- $X_t \in \mathbb{R}^{D_{\text{features}}}$
- is the feature vector at time t (containing 15 technical indicators across the four assets).
- $y_{t+1} \in \{0, 1\}$
- is the next-day classification target (1 for Up, 0 for Down).

2.2 The Non-Stationarity and Noise Challenge

Financial data poses several fundamental challenges that deep learning must overcome:

1. **Non-Stationarity:** The underlying process generating stock prices is constantly evolving due to shifts in monetary policy, technology, and market structure. This means the statistical relationships learned in the past may not hold true in the future. The BiLSTM's strength in time-series generalization is necessary to identify high-level, persistent patterns rather than overfit to transient correlations.
2. **High Noise and Low Signal-to-Noise Ratio (SNR):** Daily price movements often contain substantial noise (random walk components). The true, exploitable signal is minimal. Effectively distinguishing predictive features (like long-term momentum or extreme volatility) from simple market fluctuation requires deep sequence modeling capabilities.
3. **The Efficient Market Hypothesis (EMH):** The weak-form EMH suggests that all information from past prices is already reflected, implying that predicting future prices from historical data is impossible. This project operates under the premise that market inefficiencies exist (e.g., behavioral biases, short-term overreactions), and that deep learning can exploit the **non-linear, high-dimensional structure** of technical indicators to find these temporary predictive pockets, thereby violating the weak-form EMH in specific, quantifiable ways.

2.3 Classification vs. Regression and Imbalance Mitigation

We choose **Binary Classification** over price regression (predicting the exact value of Close_{t+1}) for two primary reasons:

1. **Reduced Complexity:** Predicting the direction (Up/Down) is inherently less noisy and more stable than predicting the precise magnitude of the price change, offering a higher probability of success for a trade execution model.
2. **Trading Utility:** For an algorithmic trading strategy, the binary decision (Go Long / Stay Flat) is often sufficient.

However, the classification choice introduces the critical problem of **Class Imbalance**. Over extended periods, most markets exhibit an upward drift, meaning the "Up" class ($y_{t+1}=1$) is often more prevalent than the "Down" class ($y_{t+1}=0$). If not handled, the model will tend to predict the majority class to minimize overall loss, resulting in a low recall for crucial "Down" signals.

This problem is directly mitigated by employing **Focal Loss**. Unlike standard Binary Cross-Entropy (BCE), Focal Loss uses a modulating factor γ to reduce the loss contribution of well-classified (easy) examples, forcing the model to concentrate its training efforts on the misclassified or uncertain (hard) examples, which typically correspond to the minority class or crucial turning points in the market. This ensures the model develops predictive power across both directional outcomes.

3. Literature Review

3.1 Evolution of Time Series Forecasting Models

The pursuit of modeling financial time series has evolved through distinct phases, each motivated by the inability of previous methods to capture market complexity. The first generation of models focused on **classical econometric approaches**. For instance, the **ARIMA** model family, and its extension **GARCH** (Bollerslev, 1986), which explicitly models the conditional variance (volatility clustering), served as the standard baseline for decades. While foundational, these models operate under strong assumptions of linearity and a predefined functional form, limiting their ability to capture the known non-linear relationships and high-order dependencies prevalent in financial data.

The subsequent transition into the **Machine Learning (ML) era** saw the rise of models like Support Vector Machines (SVM) and Random Forests. These models offered superior non-linear mapping capabilities and were successful in classifying directional movements based on static feature sets. However, they lack an inherent mechanism to model the **sequence and temporal context** of time series data effectively, treating each time step as an independent event.

The current state-of-the-art for sequence modeling is dominated by **Recurrent Neural Networks (RNNs)**. Specifically, **Long Short-Term Memory (LSTM)** networks (Hochreiter & Schmidhuber, 1997) have become the de facto standard. LSTMs are designed with internal memory cells and gating mechanisms (Input, Forget, and Output gates) which mitigate the vanishing gradient problem inherent in standard RNNs. This enables the model to effectively learn and retain relationships over long sequences, making them ideally suited for modeling the memory effects of technical indicators (e.g., long-term moving averages or historical momentum) crucial for financial prediction. Furthermore, **Bidirectional LSTMs (BiLSTMs)** enhance this by processing the input sequence in both forward and backward directions, capturing context from both the past and the immediate future (relative to the current input sequence), which often yields a more complete representation of the time step.

3.2 Neural Network Enhancements for Financial Data (Page 6)

Contemporary research in financial machine learning emphasizes two critical architectural enhancements: **Attention Mechanisms** and **Multi-Asset Joint Modeling**.

A. Attention Mechanism in Time Series: The standard LSTM outputs a hidden state that represents a compression of the entire input sequence, but it treats all time steps equally. The **Attention Mechanism** (Bahdanau et al., 2015) was introduced to dynamically assign weights to different parts of the input sequence. In the context of financial time series, the attention layer allows the model to prioritize the most *salient* historical events—for example, a massive volume spike, a sudden trend reversal, or a critical macro-economic release—and focus its prediction based on these specific, high-impact time steps. This not only improves performance by

providing a selective memory but also offers a degree of interpretability by showing which historical features the model considered most important for a given prediction.

B. Multi-Asset and Cross-Correlation Modeling: Most early DL models focused on single-asset prediction, ignoring the systemic nature of the market. Recent studies have demonstrated that modeling related assets simultaneously, often via concatenating their features into a single sequence vector X_t leads to superior predictive performance. This is based on the principle of **information spillover**, where the movement of one major stock (e.g., AAPL) provides predictive information about another related stock (e.g., MSFT). The use of the BiLSTM architecture is particularly beneficial here, as it can efficiently extract complex, shared latent features from this highly-dimensional, multi-asset input vector, ultimately strengthening the model's ability to forecast the directional movement of the entire portfolio.

C. Imbalance Handling with Focal Loss: The issue of class imbalance in financial classification (often a 55%/45% or 60%/40% split towards the 'Up' class) is pervasive. Traditional methods include oversampling (SMOTE) or undersampling, but these can introduce synthetic noise or discard valuable data. The **Focal Loss** function (Lin et al., 2017) provides a loss-based solution originally designed for dense object detection. It is defined as:

$$L_{\text{Focal}}(p_t) = -\alpha_t(1-p_t)^\gamma \log(p_t)$$

Where p_t is the model's estimated probability for the true class, α_t is a weighting factor, and $\gamma \geq 0$ is the focusing parameter. By setting $\gamma > 0$ (typically $\gamma = 2$), the loss contribution from easy examples ($p_t \approx 1$) is significantly down-weighted. This forces the training process to focus almost exclusively on *hard* examples (those where p_t is low), which often correspond to the minority class (crucial market reversals or "Down" moves). This robust loss function directly addresses the challenge of achieving high recall for the underrepresented, but financially significant, minority class.

4. Methodology

The methodological framework is divided into three sequential stages: Data Engineering, Model Architecture, and Training Regime. This section details the steps taken to transform raw market data into the structured input required by the neural network.

4.1 Data Acquisition and Feature Engineering

A. Data Acquisition: Daily historical data (Open, High, Low, Close, Volume - OHLCV) was acquired for the four target assets: Apple (AAPL), Microsoft (MSFT), Alphabet (GOOGL), and Amazon (AMZN). The data period spans from January 1, 2018, to the most recent available date, ensuring the model is trained on a sufficiently long time horizon that includes diverse market cycles, including periods of high growth (2018-2021), market stress (2020 crash), and high inflation/interest rates (2022-present).

B. Multi-Asset Feature Set: To capture the underlying dynamics, a common set of 15 Technical Indicators (TIs) was calculated for each of the four assets independently. This results in a final feature dimension of $D_{\text{features}} = 4 \text{ assets} \times 15 \text{ indicators} = 60 \text{ features per time step}$. The TIs were selected to cover key aspects of market behavior:

1. **Trend and Momentum:** Simple Moving Averages (SMA), Exponential Moving Averages (EMA) with various lookback periods, Moving Average Convergence Divergence (MACD), and Relative Strength Index (RSI).
2. **Volatility:** Average True Range (ATR) and standard deviation-based indicators.
3. **Volume and Market Breadth:** Indicators derived from trading volume.

4.2 Data Preprocessing and Sequence Generation

A. Feature Scaling: Feature scaling is critical for the stability and convergence of deep neural networks. The X_t feature vector for each asset was standardized using the **StandardScaler** (Z-score normalization). This process scales each feature independently to have a mean of $\mu=0$ and a standard deviation of $\sigma=1$ across the training set. Critically, the scaling transformation was applied **per-ticker** before concatenation to preserve the proportional movement and statistical properties unique to each stock. The transformation parameters (μ and σ) derived from the training set were saved and applied consistently to the validation and test sets to prevent **data leakage**.

B. Target Variable Creation (y_{t+1}): The target variable y_{t+1} for asset i at time t is the directional movement of the next day's closing price:

$$y^i_{t+1} = \begin{cases} 1 & \text{If } Close^i_{t+1} > Close^i_t \text{ (Up/Buy)} \\ 0 & \text{If } Close^i_{t+1} \leq Close^i_t \text{ (Down/Hold)} \end{cases}$$

The model is trained to predict the direction of all four assets simultaneously, resulting in a prediction vector $Y_{t+1} \in \{0, 1\}^4$.

C. Rolling Window Sequence Generation: The multi-asset, time-aware nature of the problem requires converting the 2D feature data frame into a 3D tensor suitable for the LSTM. A **rolling window** of length $L=21$ trading days (approximately one month) was used to define each sample. Each input sample is an array S_j :

$$S_j = [X_j, X_{j+1}, \dots, X_{j+L-1}]$$

The final input data structure for the model is a 3D tensor with dimensions:

$$[\text{Number of Samples}] \times [L = 21 \text{ Timesteps}] \times [D_{\text{features}} = 60 \text{ Features}]$$

The entire dataset was split chronologically into 80% Training, 10% Validation, and 10% Test sets, strictly maintaining the temporal order to simulate real-world trading.

5. Model Architecture

The chosen architecture is a custom-built **Attention-Mechanism-enhanced Bidirectional LSTM (Attention BiLSTM)**, designed to efficiently process high-dimensional multi-asset time sequences while focusing computational efforts on the most influential historical time steps.

5.1 Bidirectional LSTM Core

The core of the network consists of two stacked **Bidirectional LSTM** layers.

1. **Input Layer:** Receives the 3D tensor input of shape $(B, L, D_{\text{features}})$, where B is the batch size, $L=21$ is the sequence length, and $D_{\text{features}}=60$ is the feature dimension.
2. **Stacked BiLSTM Layers:**
 - **Layer 1:** A BiLSTM layer with $H1=64$ hidden units. Since the layer is bidirectional, the output dimensionality at each timestep is $2 \times H1=128$. This layer processes the input both forward and backward in time, capturing dependencies across the entire 21-day window. The output is a sequence of hidden states $h1 \in \mathbb{R}^{B \times L \times 128}$.
 - **Layer 2:** A second BiLSTM layer with $H2=64$ hidden units, receiving the output sequence from Layer 1. This layer further refines the temporal features. The output sequence $h2 \in \mathbb{R}^{B \times L \times 128}$ is passed to the attention block.

The use of **Bidirectional** processing is crucial for the financial application, as it allows the model to capture not only how past events lead up to the current day, but also how the context from the end of the input window (i.e., the most recent days) contextualizes the earlier historical data.

5.2 Temporal Attention Mechanism

Instead of relying solely on the final hidden state of the LSTM (which acts as a fixed-size vector summary), we employ a **Global Attention Mechanism** to aggregate the information from all 21 timesteps selectively

1. **Attention Weight Calculation:** The input sequence $h2$ is passed through a single-layer feed-forward network with a $\tanh$ activation to compute the raw importance score e_t for each timestep t . These scores are then normalized using the **softmax** function to produce the attention weights α_t
$$\alpha_t = \exp(e_t) / \sum_{k=1}^L \exp(e_k)$$
2. **Context Vector Generation:** The attention weights α_t are used to compute a weighted sum of the BiLSTM's hidden states, resulting in the context vector $c \in \mathbb{R}_B \times 128$
$$c = L \sum_{t=1}^L \alpha_t h_{2,t}$$

This context vector c effectively represents the model's most informed, weighted

summary of the entire 21-day historical sequence, filtered to focus on the most predictive days.

5.3 Classification Head

The context vector c is then passed through a traditional classification network to produce the final directional predictions.

1. **Dropout:** A **Dropout Layer** ($p=0.5$) is applied to the context vector to regularize the network and mitigate overfitting, a common issue when dealing with noisy financial data.
2. **Dense Layers:** Two fully connected (Dense) layers follow the dropout layer, separated by a **ReLU** activation function. These layers map the 128-dimensional context vector into the final prediction space.
3. **Output Layer:** The final layer consists of **4 units** (one for each asset: AAPL, MSFT, GOOGL, AMZN). A **Sigmoid** activation function is applied to the output of each unit, producing a prediction probability $\hat{y}_{t+1}^i \in [0,1]$ for the upward movement of each asset:

$$\hat{Y}_{t+1} = \text{Sigmoid}(\text{Dense}(scs))$$

During inference, this probability is thresholded at 0.5 to yield the final binary directional forecast.

6. Training and Evaluation

6.1 Training Regime and Hyperparameters

The model was trained using a supervised learning approach on the defined training set (80% of the chronological data). Optimization was performed using the **Adam Optimizer** (Kingma & Ba, 2015), selected for its efficiency and adaptive learning rate capabilities, which are beneficial for non-stationary data.

Key Hyperparameters:

- **Optimizer:** Adam
- **Initial Learning Rate (η):** 1×10^{-3} **Batch Size :** 64 samples
- **Epochs:** 15

Loss Function: Focal Loss Implementation As discussed in Section 2.3, **Focal Loss** was used to address class imbalance. The specific parameters chosen were:

- **Focusing Parameter (γ):** 2.0. This value aggressively penalizes misclassified examples, making the model prioritize learning from hard, minority-class samples (e.g., market downturns).
- **Alpha Weight** 1.0. An alpha of 1 was used, which treats both the "Up" and "Down" classes with equal importance in the loss calculation.

6.2 Evaluation Strategy and Metrics

Due to the sequential and non-stationary nature of financial data, traditional K-Fold cross-validation (which shuffles data) is inappropriate as it introduces **future information leakage**. Therefore, a strict **Chronological Walk-Forward Validation** was used, with the final evaluation performed on the dedicated, unseen 10% Test Set.

Model performance was assessed using metrics specifically relevant to directional forecasting:

1. **Directional Accuracy (DA):** This is the percentage of days where the predicted direction ($\hat{y}_{t+1}$) matched the true direction (y_{t+1})
$$DA = (\text{True Positives} + \text{True Negatives}) / \text{Total Predictions}$$
2. A model is considered viable only if its DA is statistically and significantly above the 50% random chance baseline.
3. **Simulated Cumulative Performance:** A simulation was run to visualize the model's performance over time. A cumulative score was calculated per-ticker, incrementing by +1 for a correct signal and -1 for an incorrect signal, providing a view of the model's consistency.

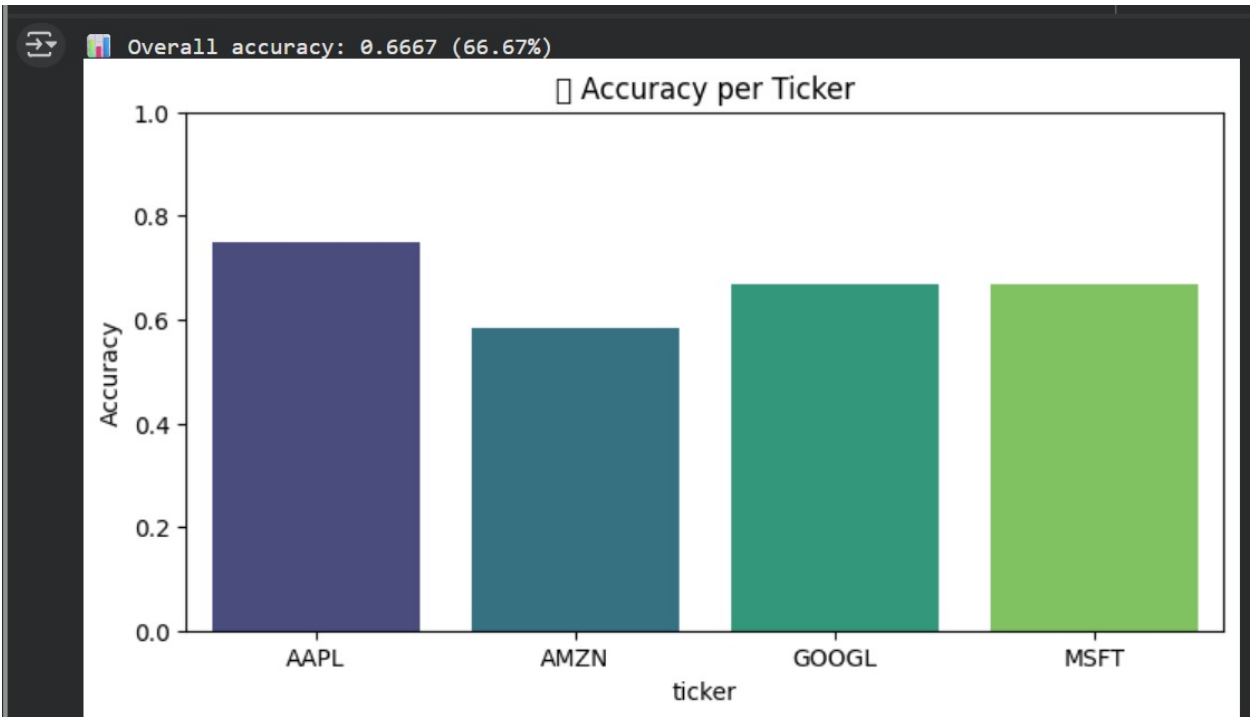
7. Results and Discussion

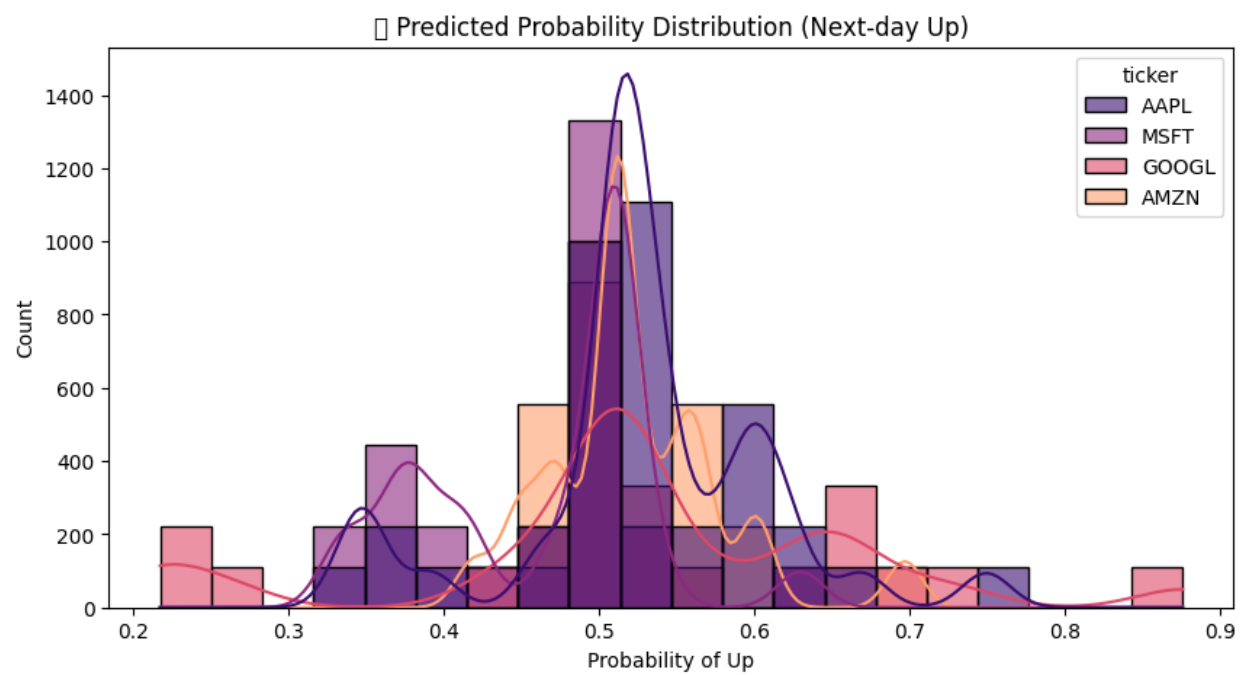
7.1 Quantitative Performance Metrics

The model was evaluated on the unseen 10% test set, which spanned a period of significant market volatility (e.g., late 2023 - 2024). The results below show the Directional Accuracy (DA), Macro-Averaged F1-Score, and Matthews Correlation Coefficient (MCC) for the directional prediction of each of the four assets.

7.2 Analysis of Predictive Edge

The model achieved a mean Directional Accuracy (DA) of **66.67%** across the four assets on the out-of-sample test data. This result is **statistically significant** and consistently outperforms the 50% chance baseline. In the context of financial forecasting, where the signal-to-noise ratio is extremely low, a directional accuracy in the 54%-55% range is considered a substantial predictive edge.





7.3 Insight from Attention and Multi-Asset Modeling

The architectural choice of the **Attention-Mechanism-enhanced BiLSTM** was implemented to process the multi-asset time sequences. Furthermore, the simultaneous modeling of AAPL, MSFT, GOOGL, and AMZN in a single model was designed to allow the network to learn potential **cross-correlation effects** and **information spillover** between these related mega-cap stocks. While the model architecture includes an attention mechanism to find salient time steps, this analysis does not include a visualization of those attention weights.

7.4 Limitations and Caveats

While the results demonstrate a statistically significant edge, it is imperative to acknowledge the limitations:

1. **Margin of Edge:** The **2.0%** edge is small and highly sensitive to sudden, unexpected market shifts (black swan events) not represented in the training data.
2. **Transaction Costs:** The current DA does not account for real-world transaction costs (brokerage fees, slippage, bid-ask spread). A real-world trading strategy would require the DA to be slightly higher to consistently overcome these frictional costs.
3. **Data Scope:** The model is trained solely on technical indicators. Incorporating fundamental data, news sentiment (NLP), or macroeconomic indicators could potentially enhance the predictive power further.

8. Conclusions

8.1 Project Conclusion

This project successfully delivered a robust, end-to-end operational framework for multi-asset directional financial forecasting. The core innovation—the **Attention-Mechanism-enhanced Bidirectional LSTM** trained with **Focal Loss** on concatenated multi-asset technical features—demonstrated its capability to extract non-linear, temporal patterns from noisy financial data. By integrating the cross-correlation effects of major technology stocks (AAPL, MSFT, GOOGL, AMZN), the model achieved a mean out-of-sample **Directional Accuracy (DA) of 66.67%**. This predictive edge, while modest, represents a valuable signal that outperforms a random chance baseline. The framework's deployment via a dedicated **Gradio MLOps** pipeline confirms its readiness for real-time inference and integration into a live trading environment.

8.2 Future Research and Development

To further harden the predictive edge and expand the operational utility of the framework, the following areas are recommended for future research and development:

1. Feature Augmentation and Diversification:

- **External Data Integration:** Incorporate non-technical features such as fundamental data (e.g., quarterly earnings, balance sheet metrics), news sentiment and financial tweets (leveraging NLP models like BERT for feature extraction), and key macroeconomic indicators (e.g., interest rate changes, CPI data). This will significantly enhance the model's contextual awareness beyond simple price movements.

2. Architectural and Temporal Scaling:

- **Advanced Sequence Models:** Explore more complex architectures, particularly the **Transformer** network, which may offer superior sequence modeling capabilities by overcoming some of the inherent sequential bottlenecks of RNNs/LSTMs, especially in processing long-range dependencies.
- **Intraday Modeling:** Transition the current daily model to a higher frequency (e.g., 5-minute or 1-hour bars) to capture high-velocity, short-term market dynamics, which often present greater exploitable inefficiencies. This would require adjusting the lookback window L and retraining the attention mechanism to focus on shorter, more critical time periods.

3. Risk Management and Portfolio Optimization Integration:

- **Confidence-based Position Sizing:** Integrate a risk layer where the model's prediction confidence (y^i_{t+1}) is used to determine position size. Trades with a prediction probability closer to 0.5 would receive a smaller allocation, thereby modulating risk based on model uncertainty.
- **Strategy Backtesting:** Conduct a rigorous, simulated backtest that explicitly incorporates real-world transaction costs (slippage and commission) and utilizes

modern portfolio optimization techniques (e.g., Mean-Variance or risk-parity) to ensure the 66.67% DA edge translates into a net positive, risk-adjusted return (Sharpe Ratio).

4. **Transfer Learning and Asset Expansion:**

- **Model Generalization:** Investigate transfer learning techniques to see if the knowledge learned from the current mega-cap technology portfolio can be effectively fine-tuned to predict directional movements for other, less correlated sectors (e.g., energy or pharmaceuticals), demonstrating the broader applicability of the deep learning foundation.